Universidad Nacional Autónoma de México
Facultad de Ciencias

Inteligencia Artificial

Laboratorio 1.3

Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 31/08/2026

Tipos de datos y operadores

Matthes (2019) explica que la filosofía de la comunidad de programadores de Python está contenida en lo que se conoce como "El Zen de Python" y esta consiste en evitar la complejidad y apuntar a la simplicidad siempre que sea posible. De esta forma, el código es más fácil de mantener para que el programador u otros puedan construir sobre el mismo código más adelante.

A su vez, de acuerdo con Lubanovic, Python es el lenguaje más popular para los cursos de introducción a la computación en las mejores universidades estadounidenses y empleado por más de dos mil empresas en la evaluación de habilidades de programación para sus aplicantes. Además, Python tiene licencia de código abierto, es decir que puede ser usado, modificado y distribuido libremente sin costo alguno. Esto ha permitido una gran cantidad de software útil en su biblioteca estándar desarrollado en distintas partes del mundo.

Leonardo Kuffo (2020) explica que, Jupyter ofrece a cualquiera poder ejecutar el código de inicio a fin sin obtener errores que se suscitan en la compilación convencional.

Además, al usar Jupyter Notebook, los resultados generados por el código en el pasado se quedan grabados y pueden consultarse sin tener que ejecutarlo nuevamente, así los profesores y alumnos pueden trabajar de una forma más didáctica. Suele usarse para tareas como procesamiento de datos, simulación numérica, modelado estadístico, visualización de datos, aprendizaje automático y mucho más.

Se ha nombrado al archivo nuevo como “Hola mundo” debido a que este es el primer ejemplo presentado por Matthes en su libro. Para ello, escribe lo siguiente en la primera línea de código:

```
print("Hola mundo Python")
```

```
Hola mundo Python
```

Posterior a esto, se procede a ejecutar la celda o línea de código, para esto se debe ir al menú “Cell” y seleccionar “Run Cells” como se muestra en la figura 3 o bien, presionar el atajo Ctrl + Enter.

```
print("Hola mundo Python")
```

```
Hola mundo Python
```

Para introducir otra línea de código sería necesario ir “Insert” y hacer click en “Insert Cell Below“. Por otro lado, si se empleó el comando Ctrl + Enter, se habrá desplegado lo siguiente:

Si en vez de presionar Ctrl + Enter, solo se presiona Enter, se realizaría el salto a la siguiente línea de código dentro de la misma celda.

El siguiente ejemplo consiste en desplegar “el Zen de Python”, para ello escribe lo siguiente en la segunda línea y ejecútalo:

```
import this
```

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

Al terminar el programa, podrás verificar que Jupyter Notebook permite estructurar el código en diferentes bloques o celdas, así es posible ejecutar cada bloque de código por separado a diferencia de otros entornos de programación en los que es necesario ejecutar todos desde la primera hasta la última línea. Esta característica le brinda al programador los beneficios de legibilidad y entendimiento del programa.

Los tipos de datos básicos de Python son los siguientes:

boolean (Booleanos, con valor Falso o Verdadero) int (números enteros, por ejemplo 10 y 9999) float (números flotantes, es decir, con puntos decimales como 3.1415) string (cadenas o secuencias de caracteres de texto) Cada tipo tiene sus propias reglas de uso y la computadora los maneja de distinto modo. Cuando se trabaja con números en Python las operaciones que pueden llevarse a cabo están descritas en la siguiente tabla:

```
mensaje = "Hola mundo Python"
print(mensaje)
```

```
Hola mundo Python
```

En esta ocasión, se agregó una variable llamada mensaje. Como ya se dijo antes, cada variable está vinculada a un valor, que es la información asociada a ella. El valor de "¡Hola mundo Python!" correspondería al tipo de dato string.

Aunque realmente no resulte apreciable, usar una variable significa más trabajo para el intérprete de Python. Cuando procesa la primera línea, asocia el mensaje de la variable con el mensaje "¡Hola mundo Python!". Y cuando se ejecuta la segunda línea, el valor asociado a la variable imprime el mensaje en la pantalla.

Hay casos en los que es necesario imprimir más de una sola línea, para esto en vez de utilizar comillas (“”) es necesario emplear triple apóstrofo (‘’’) como se ilustra en el siguiente ejemplo:

```
poema = ''' << Poesía, verdad de todo sueño,
nunca he sido de ti más corto dueño
que en este amo en cuyas nubes muero. >> Carlos Pellicer.'''
print(poema)
```

```
 << Poesía, verdad de todo sueño,
nunca he sido de ti más corto dueño
que en este amo en cuyas nubes muero. >> Carlos Pellicer.
```

El siguiente ejemplo sirve para ilustrar el uso del tipo de datos int (números enteros) y float (flotantes), que consiste en un programa que calcula el área de un triángulo empleando los operadores de multiplicación (*), división con punto flotante o decimal (/) y de enteros (//), es decir truncada:

```
base = 3
altura = 7
area_triangulo = (base*altura)/2
print(area_triangulo)
```

```
area_triangulo = ((base*altura)//2)
print(area_triangulo)
```

```
10.5
10
```

Desafio:

Crea un nuevo archivo en Jupyter Notebook.

Con lo aprendido sobre los tipos de datos numéricos (enteros y flotantes) y strings, diseña un único programa que realice lo siguiente: Traduce “el Zen de Python” por tu cuenta y por medio de strings despliégalo en la salida del programa.

Elabora una calculadora de tiempo siguiendo estos pasos:

1. Asignar a una variable llamada segundos_por_hora las operaciones correspondientes para calcular dicho valor.
2. Utiliza la variable anterior para calcular cuantos segundos hay en un día y asignarlo a la variable segundos_por_dia.
3. Divide segundos_por_dia entre segundos_por_hora empleando la operación de división con punto flotante(/).
4. Divide segundos_por_dia entre segundos_por_hora empleando la operación de división de enteros(//).
5. Imprime los resultados para cada uno de los puntos anteriores (para el caso de las divisiones puedes realizar la operación directamente dentro de la misma función print).
6. Analiza los resultados y comprueba que sean correctos.

```
zen_traducido = """
El Zen de Python, por Tim Peters

Lo bello es mejor que lo feo.
Lo explícito es mejor que lo implícito.
Lo simple es mejor que lo complejo.
Lo complejo es mejor que lo complicado.
Lo plano es mejor que lo anidado.
Lo disperso es mejor que lo denso.
La legibilidad cuenta.
Los casos especiales no son lo suficientemente especiales como para romper las reglas.
Aunque la practicidad vence a la pureza.
Los errores nunca deben pasar en silencio.
A menos que se silencien explícitamente.
Frente a la ambigüedad, rechaza la tentación de adivinar.
Debe haber una —y preferiblemente solo una— manera obvia de hacerlo.
Aunque esa manera puede no ser obvia al principio, a menos que seas holandés.
Ahora es mejor que nunca.
Aunque nunca es a menudo mejor que ahora mismo.
Si la implementación es difícil de explicar, es una mala idea.
Si la implementación es fácil de explicar, puede ser una buena idea.
Los espacios de nombres son una gran idea — ¡hagamos más de eso!
"""

print(zen_traducido)



# Calculadora de Tiempo

# 1. Calcular segundos por hora
segundos_por_hora = 60 * 60

print(segundos_por_hora)

# 2. Calcular segundos por día
segundos_por_dia = segundos_por_hora * 24
print(segundos_por_dia)

# 3. División con punto flotante (/)
divFlotante = segundos_por_dia / segundos_por_hora
print(divFlotante)

# 4. División entera (//)
divEnteros = segundos_por_dia // segundos_por_hora
print(divEnteros)
```

```
El Zen de Python, por Tim Peters
```

```
Lo bello es mejor que lo feo.
Lo explícito es mejor que lo implícito.
Lo simple es mejor que lo complejo.
Lo complejo es mejor que lo complicado.
Lo plano es mejor que lo anidado.
Lo disperso es mejor que lo denso.
La legibilidad cuenta.
Los casos especiales no son lo suficientemente especiales como para romper las reglas.
Aunque la practicidad vence a la pureza.
Los errores nunca deben pasar en silencio.
A menos que se silencien explícitamente.
Frente a la ambigüedad, rechaza la tentación de adivinar.
Debe haber una —y preferiblemente solo una— manera obvia de hacerlo.
Aunque esa manera puede no ser obvia al principio, a menos que seas holandés.
Ahora es mejor que nunca.
Aunque nunca es a menudo mejor que ahora mismo.
Si la implementación es difícil de explicar, es una mala idea.
Si la implementación es fácil de explicar, puede ser una buena idea.
Los espacios de nombres son una gran idea — ¡hagamos más de eso!

3600
86400
24.0
24
```

```
# Solicitamos al usuario que ingrese los segundos en una hora
segundos_por_hora = float(input("Ingrese la cantidad de segundos en una hora: "))

# Calculamos los segundos en un día
segundos_por_dia = segundos_por_hora * 24

# Realizamos la división con punto flotante
resultado_division_flotante = segundos_por_dia / segundos_por_hora

# Realizamos la división de enteros
resultado_division_enteros = segundos_por_dia // segundos_por_hora

# Imprimimos los resultados
print("Segundos en un día (cálculo directo):", segundos_por_dia)
print("División con punto flotante:", resultado_division_flotante)
print("División de enteros:", resultado_division_enteros)
resultado_division_enteros =  segundos_por_dia // segundos_por_hora
resultado_division_enteros
```

```
Ingrese la cantidad de segundos en una hora: 1200
Segundos en un día (cálculo directo): 28800.0
División con punto flotante: 24.0
División de enteros: 24.0
24.0
```

Cerrar

```
# Conclusiones

Me parecio muy interesante que JupiterNotebook pueda guardar o reg
compilaciones anteriores. Creo que es una buena idea para poder ob
avance de los alumnos durante la materia, no sabia que se podia en
que, para un futuro, seria bueno para poder guiar. Me gustaria exp
mas
```

Conclusiones

Me parecio muy interesante que JupiterNotebook pueda guardar o registrar compilaciones anteriores. Creo que es una buena idea para poder observar el avance de los alumnos durante la materia, no sabia que se podia entonces creo que, para un futuro, seria bueno para poder guiar. Me gustaria explorarla mucho mas

© 2026 UNAM Facultado de Ciencias – Todos los derechos reservados